Lezione 6

Design Patterns: Creational e Structural

Simone Colli

15 dicembre 2025

Indice

1. Contenuto della lezione
2. Perché i Design Patterns?
3. Analisi code smells
4. Creational Pattern: DTO
5. Creational Pattern: Builder
6. Structural Pattern: Adapter
7. Confronto Prima/Dopo
8. Esercizi
9. Conclusioni

Contenuto della lezione

- Analisi code smells della Lezione 5
- Creational Patterns:
  - DTO (Data Transfer Object)
  - Builder Pattern (Frontend e Backend)
- Structural Patterns:
  - Adapter Pattern
- Refactoring guidato del codice esistente
- Esempi pratici e confronti prima/dopo

Cosa sono i Design Patterns?

I **Design Patterns** sono soluzioni collaudate a problemi ricorrenti nella progettazione del software.

Caratteristiche principali

- **Riutilizzabili**: applicabili in diversi contesti
- **Documentati**: nomi e semantica condivisi
- **Collaudati**: testati nel tempo da migliaia di sviluppatori
- **Indipendenti dal linguaggio**: concetti applicabili ovunque

NON sono:

- Codice pronto da copiare
- La soluzione a tutti i problemi
- Obbligatori in ogni situazione

Categorie di Design Patterns

I pattern si dividono in tre categorie principali:

Creational Come creare oggetti in modo flessibile ed efficiente
- *Es: Builder, Factory, Singleton*

Structural Come comporre oggetti e classi in strutture più grandi
- *Es: Adapter, Decorator, Facade*

Behavioral Come gestire algoritmi e responsabilità tra oggetti
- *Es: Observer, Strategy, Command*

Oggi: Creational (DTO, Builder) e Structural (Adapter)

Code smells della Lezione 5

Prima di applicare i pattern, identifichiamo i **code smells** (segnali che il codice può essere migliorato).

Problemi identificati

1. **Validazione duplicata**: controlli sparsi in più punti
2. **Costruzione payload manuale**: codice ripetitivo
3. **Conversione dati sparsa**: snake_case $\leftrightarrow$ camelCase ovunque
4. **Risposte API inconsistenti**: formato variabile

Obiettivo: Refactoring per centralizzare logica comune.

Code Smell #1: Validazione duplicata

Problema: La validazione è ripetuta in più punti.

```
// in handleAddTask()
if (!taskData.title || taskData.title.trim() === '') {
    this.appView.showAlert('Il titolo e` obbligatorio', 'error');
    return;
}

// in handleUpdateTask()
if (!taskData.title || taskData.title.trim() === '') {
    this.appView.showAlert('Il titolo e` obbligatorio', 'error');
    return;
}

// backend - createTask()
if (empty($data['title'])) {
    $this->sendError("Il campo 'title' e` obbligatorio", 400);
    return;
}

// backend - updateTask()
if (empty($data['title'])) {
    $this->sendError("Il campo 'title' e` obbligatorio", 400);
    return;
}
```

Problema: Se cambiano le regole, dobbiamo modificare 4+ punti diversi.

Code Smell #2: Costruzione payload manuale

Problema: Costruzione oggetti ripetitiva e soggetta a errori.

```
// metodo save()
const payload = {
    title: task.title,
    description: task.description,
    due_date: task.dueDate,
    priority: task.priority,
    status: task.status
};

// metodo update()
const payload = {
    title: task.title,
    description: task.description,
    due_date: task.dueDate,      // ripetuto!
    priority: task.priority,
    status: task.status
};
```

problemi:

- ripetizione codice
- facile dimenticare campi
- nessuna validazione prima dell'invio

Code Smell #3: Conversione dati sparsa

Problema: Conversione snake_case ↔ camelCase ripetuta ovunque.

```
// in load()
const tasks = response.data.map(taskData =>
  App.Task.fromDTO({
    id: taskData.id,
    title: taskData.title,
    dueDate: taskData.due_date,    // conversione manuale
    // ...
  })
);

// in save()
return App.Task.fromDTO({
    id: data.id,
    title: data.title,
    dueDate: data.due_date,        // ripetuto!
    // ...
});

// in delete(), update(), ... (ripetuto ancora!)
```

Problema: 4-5 punti diversi con la stessa logica di conversione.

DTO - Data Transfer Object

DTO è un pattern che definisce un oggetto per trasportare dati tra layer dell'applicazione.

Responsabilità

- **Incapsulare** i dati in un unico oggetto
- **Validare** i dati prima dell'uso
- **Definire** il contratto dei dati
- **Centralizzare** le regole di validazione

DTO - Implementazione frontend

```
// services/frontend/src/js/dto/TaskDTO.js
class TaskDTO {
    constructor(data = {}) {
        this.id = data.id || null;
        this.title = data.title || '';
        this.description = data.description || '';
        this.dueDate = data.dueDate || null;
        this.priority = data.priority || 'medium';
        this.status = data.status || 'incomplete';
    }
    validate() {
        const errors = [];
        if (!this.title || this.title.trim() === '') {
            errors.push('Il titolo e` obbligatorio');
        }
        if (this.title && this.title.length > 100) {
            errors.push('Il titolo non puo` superare 100 caratteri');
        }
        return { valid: errors.length === 0, errors };
    }
}
```

DTO - Utilizzo

Prima (senza DTO):

```
if (!taskData.title || taskData.title.trim() === '') {
    alert('Errore!');
    return;
}
const task = new Task(taskData);
```

Dopo (con DTO):

```
const dto = new TaskDTO(taskData);
const validation = dto.validate();

if (!validation.valid) {
    alert(validation.errors.join(', '));
    return;
}

const task = Task.fromDTO(dto);
```

DTO - Implementazione Backend

```php
// services/backend/src/dto/TaskDTO.php
class TaskDTO {
    public $id;
    public $title;
    public $description;
    public $due_date;
    public $priority;
    public $status;

    public function __construct($data) {
        $this->id = $data['id'] ?? null;
        $this->title = $data['title'] ?? '';
        $this->description = $data['description'] ?? '';
        $this->due_date = $data['due_date'] ?? null;
        $this->priority = $data['priority'] ?? 'medium';
        $this->status = $data['status'] ?? 'incomplete';
    }

    public function validate() {
        $errors = [];
        if (empty($this->title)) {
            $errors[] = "Il campo 'title' e` obbligatorio";
        }
        return ['valid' => count($errors) === 0, 'errors' => $errors];
    }
}
```

Builder Pattern

Il **Builder** costruisce oggetti complessi passo dopo passo, separando costruzione da rappresentazione.

Quando usarlo?

- Oggetti con **molti parametri** (alcuni opzionali)
- Necessità di **validare** durante la costruzione
- Costruzione in **più step**
- Interfaccia **fluente** (method chaining)

Vantaggi:

- Codice più leggibile (`.setTitle().setPriority()`)
- Validazione automatica al `.build()`
- Immutabilità dell'oggetto finale

Builder - Senza Builder (Lezione 5)

Costruzione manuale del payload:

```
// ripetuto in save(), update(), ...
const payload = {
    title: task.title,
    description: task.description,
    due_date: task.dueDate,
    priority: task.priority,
    status: task.status
};

// nessuna validazione!
$.ajax({
    url: this.baseUrl,
    method: 'POST',
    data: JSON.stringify(payload),
    // ...
});
```

Problemi: Ripetizione, nessuna validazione, facile dimenticare campi.

Builder - Implementazione frontend

```
// services/frontend/src/js/patterns/builders/ClientRequestBuilder.js
class ClientRequestBuilder {
    constructor() {
        this.dto = new App.TaskDTO();
    }
    setTitle(title) {
        this.dto.title = title;
        return this; // method chaining
    }
    setDescription(description) {
        this.dto.description = description;
        return this;
    }
    setDueDate(dueDate) {
        this.dto.dueDate = dueDate;
        return this;
    }
    build() {
        const validation = this.dto.validate();
        if (!validation.valid) {
            throw new Error(validation.errors.join(', '));
        }
        return { ...this.dto }; // immutabile
    }
}
```

Builder - Utilizzo frontend

Dopo (con Builder):

```
const request = new ClientRequestBuilder()
    .setTitle('Studiare Design Patterns')
    .setDescription('Completare esercizi')
    .setDueDate('2025-12-20')
    .setPriority('high')
    .setStatus('incomplete')
    .build(); // valida automaticamente!

$.ajax({
    url: this.baseUrl,
    method: 'POST',
    data: JSON.stringify(request),
    // ...
});
```

Benefici: Fluente, validato, leggibile, difficile sbagliare.

Builder - Implementazione Backend

```php
// services/backend/src/patterns/builders/ServerResponseBuilder.php
class ServerResponseBuilder {
    private $statusCode = 200;
    private $data = null;
    private $message = '';
    private $metadata = [];

    public function success($code = 200) {
        $this->statusCode = $code;
        return $this;
    }

    public function withData($data) {
        $this->data = $data;
        return $this;
    }

    public function withMessage($message) {
        $this->message = $message;
        return $this;
    }

    public function send() {
        http_response_code($this->statusCode);
        echo json_encode([
            'success' => true,
            'version' => '1.0',
            'data' => $this->data,
            'message' => $this->message,
            'metadata' => array_merge(['timestamp' => date('Y-m-d H:i:s')], $this->metadata)
        ]);
    }
```

Builder - Utilizzo backend

Prima (Lezione 5):

```
http_response_code(201);
echo json_encode([
    'success' => true,
    'data' => $task
]);
```

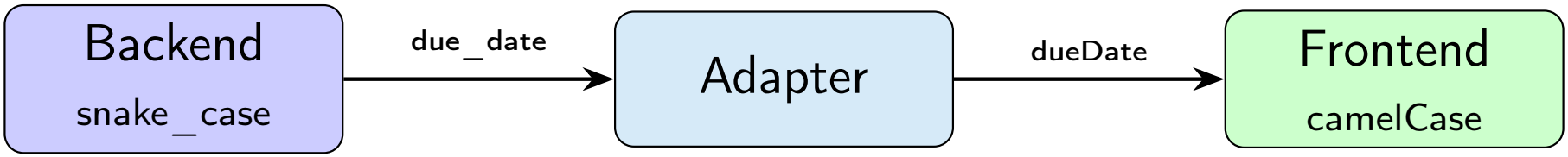
Dopo (con Builder):

```
// versione factory method (breve)
ServerResponseBuilder::created($task, 'Task creata')->send();

// versione fluente (completa)
(new ServerResponseBuilder())
    ->success(201)
    ->withData($task)
    ->withMessage('Task creata con successo')
    ->send();
```

Adapter Pattern

L'**Adapter** converte l'interfaccia di una classe in un'altra attesa dai client.



Quando usarlo?

- Interfacce **incompatibili** (es. snake_case vs camelCase)
- Integrazione con **librerie esterne**
- Conversione **formati** diversi

Adapter - Problema da risolvere

Backend (PHP, snake_case):

```
{
    "id": 1,
    "title": "Task",
    "due_date": "2025-12-20",
    "created_at": "2025-12-15"
}
```

Frontend (JS, camelCase):

```
{
    id: 1,
    title: "Task",
    dueDate: "2025-12-20",
    createdAt: "2025-12-15"
}
```

Problema: Conversione sparsa in 4-5 punti diversi nel codice.

Adapter - Implementazione

```
// services/frontend/src/js/patterns/adapters/ResponseAdapter.js
class ResponseAdapter {
    // server (snake_case) $\to$ client (camelCase)
    toTaskDTO(serverData) {
        return new App.TaskDTO({
            id: serverData.id,
            title: serverData.title,
            description: serverData.description,
            dueDate: serverData.due_date,        // conversione
            priority: serverData.priority,
            status: serverData.status,
            createdAt: serverData.created_at,    // conversione
            updatedAt: serverData.updated_at     // conversione
        });
    }

    toTask(serverData) {
        const dto = this.toTaskDTO(serverData);
        return App.Task.fromDTO(dto);
    }

    toTaskArray(serverDataArray) {
        return serverDataArray.map(data => this.toTask(data));
    }
}
```

Adapter - Utilizzo

Prima (Lezione 5, ripetuto ovunque):

```
// in load()
const tasks = response.data.map(taskData =>
  App.Task.fromDTO({
    id: taskData.id,
    dueDate: taskData.due_date,  // ripetuto
    // ...
  })
);

// in save()
return App.Task.fromDTO({
    id: data.id,
    dueDate: data.due_date,      // ripetuto
    // ...
});
```

Dopo (con Adapter, un solo punto):

```
const adapter = new ResponseAdapter();

// in load()
const tasks = adapter.toTaskArray(response.data);

// in save()
return adapter.toTask(data);
```

Adapter - RequestAdapter

Anche per le richieste (Client → Server):

```
// services/frontend/src/js/patterns/adapters/RequestAdapter.js
class RequestAdapter {
    // client (camelCase) $\to$ server (snake_case)
    fromTaskDTO(dto) {
        return {
            id: dto.id,
            title: dto.title,
            description: dto.description,
            due_date: dto.dueDate,      // conversione
            priority: dto.priority,
            status: dto.status
        };
    }
}
```

Utilizzo:

```
const requestAdapter = new RequestAdapter();
const payload = requestAdapter.fromTaskDTO(dto);
// invia al server
```

Confronto Lezione 5 vs Lezione 6

Aspetto	Lezione 5	Lezione 6
Validazione	Sparsa, duplicata (4+ punti)	Centralizzata (DTO)
Costruzione richieste	Manuale, ripetitiva	Builder (fluente, validata)
Conversione dati	Sparsa (4-5 punti)	Adapter (1 punto)
Risposte API	Inconsistenti	Builder (standardizzate)
Testabilità	Difficile	Facile (pattern isolati)
Manutenibilità	Bassa	Alta

Costo: Complessità iniziale maggiore
Beneficio: Codice più robusto, scalabile e professionale

Quando NON usare i pattern

I pattern hanno un **costo** (complessità). Non usarli quando:

- **Problema troppo semplice**: validare un solo campo non serve un DTO
- **Costruttore con 2-3 parametri**: non serve un Builder
- **Progetto piccolo/prototipo**: overhead inutile
- **Solo per "sembrare professionali"**: pattern-driven development è sbagliato

Regola d'oro: Prima scrivi codice funzionante semplice, poi refactoring se serve.

Quando usarli:

- Il problema si ripete (DRY - Don't Repeat Yourself)
- Il codice diventa difficile da mantenere
- Necessità di testabilità e scalabilità

Esercizi proposti

Esercizio 1 - Estendere DTO:

- Aggiungere campo `assignee: string` al DTO
- Aggiungere validazione: "assignee deve essere un'email valida"
- Testare con Builder e Adapter

Esercizio 2 - Nuovo Builder Method:

- Aggiungere metodo `.setTags(tags: string[])` al Builder
- Validare: massimo 5 tag
- Integrare con l'Adapter

Esercizio 3 - Factory Methods:

- Creare `ServerResponseBuilder.validationFailed(errors)`
- Creare `ServerResponseBuilder.unauthorized(message)`
- Usarli nel TaskGateway

Riepilogo

Pattern applicati oggi:

- **DTO**: Validazione centralizzata, contratto chiaro
- **Builder (FE)**: Costruzione richieste fluente e validata
- **Builder (BE)**: Risposte HTTP consistenti
- **Adapter**: Conversione centralizzata snake_case $\leftrightarrow$ camelCase

Benefici ottenuti:

- Codice più robusto e testabile
- Manutenibilità migliorata
- Meno duplicazione
- Separazione delle responsabilità

Prossima lezione: Behavioral Patterns (Observer, Strategy)